complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() transform & group

filter() null checks + collect() transform & group

filter() comparisons + collect() transform & group

filter() method calls returning boolean + collect() transform & group

distinct() rely on equals() / hashcode() + collect() transform & group

distinct() immutable objects + collect() transform & group

map() transform objects + collect() transform & group

map() create new objects + collect() transform & group

map() call pure functions + collect() transform & group

flatmap() return stream + collect() transform & group

flatmap() split data + collect() transform & group

flatmap() expand collections + collect() transform & group

sorted() natural ordering + collect() transform & group

sorted() custom comparator + collect() transform & group

sorted() multi-field compare + collect() transform & group

peek() logging + collect() transform & group

peek() debugging + collect() transform & group

peek() metrics + collect() transform & group

Perfect — this time **every example explicitly does BOTH**:

1. **Transform** (via map, mapping, or downstream collectors)
2. **Group** (via Collectors.groupingBy(...))

Each item is a **complete, standalone Java class**, Java 8+, clean and interview-ready.

---

1 filter() boolean checks + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanTransformGroup {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      Map<String, List<Integer>> result =
            numbers.stream()
                  .filter(n -> n % 2 == 0)            // boolean check
                  .map(n -> n * 2)                    // transform
                  .collect(Collectors.groupingBy(
                        n -> n > 30 ? "HIGH" : "LOW"
                  ));

      System.out.println(result);
   }
}
```

---

2 filter() null checks + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterNullTransformGroup {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit", null, "Bharti");

      Map<Integer, List<String>> result =
            names.stream()
                  .filter(Objects::nonNull)           // null check
                  .map(String::toUpperCase)           // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

3 filter() comparisons + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonTransformGroup {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      Map<String, List<Integer>> result =
            salaries.stream()
                  .filter(s -> s >= 30000)              // comparison
                  .map(s -> s + 5000)                   // transform
                  .collect(Collectors.groupingBy(
                        s -> s >= 50000 ? "SENIOR" : "MID"
                  ));

      System.out.println(result);
   }
}
```

4 filter() method call boolean + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodTransformGroup {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Ravi", false),
            new User("Amit", true)
      );

      Map<Boolean, List<String>> result =
            users.stream()
                  .filter(User::isActive)              // method call
                  .map(u -> u.name.toUpperCase())        // transform
                  .collect(Collectors.groupingBy(
                        name -> name.startsWith("A")
                  ));

      System.out.println(result);
   }
}
```

---

5 distinct() equals/hashCode + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class DistinctEqualsTransformGroup {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      @Override
      public boolean equals(Object o) {
         return o instanceof Person p && name.equals(p.name);
      }

      @Override
      public int hashCode() {
         return Objects.hash(name);
      }
   }

   public static void main(String[] args) {
      List<Person> people = List.of(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Map<Integer, List<String>> result =
            people.stream()
                  .distinct()
                  .map(p -> p.name)                    // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

6 distinct() immutable objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class DistinctImmutableTransformGroup {
   public static void main(String[] args) {
      List<String> values = List.of("Java", "Java", "Python", "Go");

      Map<Character, List<String>> result =
            values.stream()
                  .distinct()
                  .map(String::toUpperCase)            // transform
                  .collect(Collectors.groupingBy(s -> s.charAt(0)));

      System.out.println(result);
   }
}
```

---

7 map() transform objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapTransformGroup {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit", "ravi");

      Map<Integer, List<String>> result =
            names.stream()
                  .map(String::toUpperCase)            // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

---

8 map() create new objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapNewObjectTransformGroup {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit", "Ravi");

      Map<Character, List<UserDTO>> result =
            names.stream()
                  .map(UserDTO::new)                    // new object
                  .collect(Collectors.groupingBy(
                        dto -> dto.name.charAt(0)
                  ));

      System.out.println(result);
   }
}
```

---

9 map() pure functions + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionTransformGroup {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      Map<Boolean, List<Integer>> result =
            numbers.stream()
                  .map(MapPureFunctionTransformGroup::square)
                  .collect(Collectors.groupingBy(
                        n -> n > 10
                  ));

      System.out.println(result);
   }
}
```

---

10 flatMap() return Stream + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamTransformGroup {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      Map<Boolean, List<Integer>> result =
            lists.stream()
                  .flatMap(List::stream)
                  .map(n -> n * 10)                  // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println(result);
   }
}
```

---

11 flatMap() split data + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitTransformGroup {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");

      Map<Integer, List<String>> result =
            sentences.stream()
                  .flatMap(s -> Arrays.stream(s.split(" ")))
                  .map(String::toLowerCase)            // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

---

12 flatMap() expand collections + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandTransformGroup {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      Map<Integer, List<String>> result =
            data.values().stream()
                  .flatMap(Collection::stream)
                  .map(String::toUpperCase)            // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

---

13 sorted() natural order + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalTransformGroup {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      Map<Boolean, List<Integer>> result =
            numbers.stream()
                  .sorted()
                  .map(n -> n * 2)                    // transform
                  .collect(Collectors.groupingBy(n -> n > 5));

      System.out.println(result);
   }
}
```

---

14 sorted() custom comparator + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomTransformGroup {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      Map<Character, List<String>> result =
            names.stream()
                  .sorted(Comparator.reverseOrder())
                  .map(String::toUpperCase)              // transform
                  .collect(Collectors.groupingBy(s -> s.charAt(0)));

      System.out.println(result);
   }
}
```

---

15 sorted() multi-field + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldTransformGroup {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      Map<String, List<Integer>> result =
            employees.stream()
                  .sorted(Comparator
                        .comparing((Employee e) -> e.name)
                        .thenComparingInt(e -> e.age))
                  .map(e -> e.age)                    // transform
                  .collect(Collectors.groupingBy(
                        age -> age >= 30 ? "SENIOR" : "JUNIOR"
                  ));

      System.out.println(result);
   }
}
```

---

16 peek() logging + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingTransformGroup {
   public static void main(String[] args) {
      Map<Boolean, List<Integer>> result =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Log: " + n))
                  .map(n -> n * 10)                    // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println(result);
   }
}
```

---

17 peek() debugging + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugTransformGroup {
   public static void main(String[] args) {
      Map<Boolean, List<Integer>> result =
            List.of(10, 20, 30).stream()
                  .peek(n -> {
                     if (n == 20) System.out.println("Debug hit: " + n);
                  })
                  .map(n -> n + 1)                     // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println(result);
   }
}
```

---

18 peek() metrics + collect() (Transform & Group)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsTransformGroup {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      Map<Boolean, List<Integer>> result =
            List.of(5, 10, 15).stream()
                  .peek(n -> counter.incrementAndGet())
                  .map(n -> n * 2)                    // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println("Processed: " + counter.get());
      System.out.println(result);
   }
}
```